25dce049 department of computer engineering

Practical – 5

Registration Form with Frontend Validation and User-Friendly Error Handling

Problem Definition

Create a student registration form for the StudentHub portal using HTML5 input types and JavaScript validation for name, email, mobile number, password, confirm password, course, year, gender, and terms acceptance. Use regular expressions for validation, show errors next to each field, display a password strength meter, and keep the form usable with keyboard and screen readers. Use valid and invalid sample registration inputs as test data.

A. Theory

1. Are correct input types and attributes used?

Answer: Yes. Each field uses the input type that matches its data: type="text" for the name, type="email" for the email address, type="tel" with inputmode="numeric" for the mobile number, and type="password" for both password fields. Course and year use <select>, gender uses a group of radio buttons, and the terms use a checkbox. Attributes such as required, minlength, maxlength, pattern, autocomplete, placeholder, and aria-describedby are added so that browsers, password managers, mobile keyboards, and screen readers can help the user. The form has the novalidate attribute so that the browser's default pop-up bubbles are replaced by our own consistent messages; the HTML5 attributes still document the rules and still work as a fallback in the markup.

2. Are validation errors displayed near the relevant fields?

Answer: Yes. Every field has a <p class="error-msg"> directly below it, connected to the input with aria-describedby. When validation fails the message is written into that element, the input receives aria-invalid="true", and a red border and a warning sign appear so that the problem is not shown by colour alone. The message is inside an aria-live="polite" region, so screen readers announce it. On submit, an error summary at the top lists every problem with a link to the field, and focus moves to the summary.

3. Is password strength checked?

Answer: Yes. The function checkPassword() tests five rules with regular expressions (at least 8 characters, lowercase, uppercase, digit, special character), adds a bonus for 12 or more characters, and subtracts points for repeated characters (aaa) and common words (password, qwerty, 123456). The score is converted to Weak, Fair, Good, or Strong and shown as a coloured bar, as text, and as a live checklist of rules marked Met or Not met. A password is accepted only when all five rules are satisfied and no common word is present.

4. Is the form accessible using keyboard and screen-reader-friendly labels?

Answer: Yes. Every control has a visible <label> linked with for/id; the gender radio buttons are wrapped in <fieldset> and <legend>; the tab order follows the visual order; focus outlines are visible; a skip link is provided; required fields are marked with * and the required attribute; error messages are announced through aria-live; the password can be revealed with a real <button> that uses aria-pressed; and the whole form can be completed without a mouse.

5. What is a regular expression?

Answer: A regular expression (regex) is a pattern that describes a set of strings. In JavaScript it is written between slashes, for example /^[6-9]\d{9}$/, and is tested with regex.test(text), which returns true or false. Regex is ideal for checking the format of names, emails, phone numbers, and passwords.

6. What do the anchors ^ and $ and the quantifier {n} mean?

Answer: ^ matches the start of the text and $ matches the end, so the whole value must follow the pattern and not just a part of it. A quantifier such as {10} means exactly ten times, {2,5} means two to five times, + means one or more, * means zero or more, and ? means optional.

7. What is the difference between test(), match(), and exec()?

Answer: regex.test(str) only answers true or false and is the best choice for validation. str.match(regex) and regex.exec(str) return the matched text and capture groups (or null), which is useful when the matched pieces are needed.

8. What is the difference between the input, change, blur, and keyup events?

Answer: input fires on every change of the value (typing, pasting, voice). keyup fires when a key is released. change fires when the value is committed (on leaving a text box, or immediately for select, radio, and checkbox). blur fires when the field loses focus. This practical validates on blur first, and then on every input event so that the message disappears as soon as the user fixes the mistake.

9. Why is client-side validation not enough?

Answer: Client-side validation gives quick feedback but can be bypassed by disabling JavaScript or sending a request directly. The server must therefore validate and sanitise the same data again. This is done in Practical 7 with PHP.

10. How does a canvas CAPTCHA work and what are its limits?

Answer: The script generates a random code, draws it on a <canvas> with rotation, noise lines, and dots, and compares the typed answer with the stored code. It is a learning demonstration: because the code exists in the browser, a real system must generate and verify a CAPTCHA on the server. Image CAPTCHAs are also hard for some users, so the alternative of a refresh button and an accessible service should be offered in real products.

B. Implementation / Output / Result

Extend the StudentHub project from Practical 4. Replace register.html, append the new styles to css/style.css, and add js/register.js. The shared js/script.js from Practical 4 is also linked so that the theme switcher and menu continue to work on this page.

1. Project folder structure

```
StudentHub/
├── index.html
├── about.html
├── register.html    (replaced with the validated registration form)
├── dashboard.html
├── events.html
├── contact.html
├── feedback.html
├── css/
│   └── style.css    (Practical 4 styles + Practical 5 additions)
└── js/
    ├── script.js     (Practical 4)
    └── register.js   (new)
```

2. Registration page (save as register.html)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>StudentHub | Student Registration</title>
  <link rel="stylesheet" href="css/style.css">
  <script src="js/script.js" defer></script>
  <script src="js/register.js" defer></script>
</head>
<body>
  <a class="skip-link" href="#main-content">Skip to main content</a>

  <header class="site-header">
    <div class="container header-row">
      <p class="brand"><strong>StudentHub</strong></p>
      <div class="header-controls">
        <button class="icon-btn" id="theme-toggle" type="button" aria-label="Dark theme" aria-
pressed="false">Dark</button>
        <button class="icon-btn menu-toggle" id="menu-toggle" type="button" aria-label="Toggle
navigation menu" aria-expanded="false" aria-controls="main-nav">&#9776;</button>
      </div>
      <nav class="main-nav" id="main-nav" aria-label="Main navigation">
        <ul class="nav-list">
          <li><a href="index.html">Home</a></li>
          <li><a href="about.html">About</a></li>
          <li><a href="register.html" aria-current="page">Register</a></li>
          <li><a href="dashboard.html">Dashboard</a></li>
          <li><a href="events.html">Events</a></li>
          <li><a href="contact.html">Contact</a></li>
          <li><a href="feedback.html">Feedback</a></li>
        </ul>
      </nav>
    </div>
  </header>

  <main id="main-content" class="container">
    <section class="panel form-panel" aria-labelledby="reg-title">
      <h1 id="reg-title">Student Registration</h1>
      <p class="muted">Fields marked <span aria-hidden="true">*</span><span class="visually-
hidden">with an asterisk</span> are required.</p>

      <!-- Error summary (filled by JavaScript on a failed submit) -->
      <div class="error-summary" id="error-summary" role="alert" tabindex="-1" hidden>
        <h2 id="error-summary-title">Please correct the following</h2>
        <ul id="error-list"></ul>
      </div>

      <!-- Success message -->
      <div class="alert success" id="success-box" role="status" tabindex="-1" hidden></div>
```

```
      <form id="register-form" novalidate autocomplete="on">

        <div class="field">
          <label for="fullname">Full name <span class="req" aria-hidden="true">*</span></label>
          <input type="text" id="fullname" name="fullname" autocomplete="name" required
                 minlength="2" maxlength="50" pattern="[A-Za-z][A-Za-z .'\-]{1,49}"
                 placeholder="e.g. Riya Patel" aria-describedby="fullname-hint fullname-error">
          <p class="hint" id="fullname-hint">Letters, spaces, dots, apostrophes and hyphens
only.</p>
          <p class="error-msg" id="fullname-error" aria-live="polite"></p>
        </div>

        <div class="field">
          <label for="email">Email address <span class="req" aria-
hidden="true">*</span></label>
          <input type="email" id="email" name="email" autocomplete="email" required
maxlength="254"
                 placeholder="name@example.com" aria-describedby="email-hint email-error">
          <p class="hint" id="email-hint">Example: riya.patel@example.com</p>
          <p class="error-msg" id="email-error" aria-live="polite"></p>
        </div>

        <div class="field">
          <label for="mobile">Mobile number <span class="req" aria-
hidden="true">*</span></label>
          <input type="tel" id="mobile" name="mobile" inputmode="numeric" autocomplete="tel-
national" required
                 maxlength="10" pattern="[6-9][0-9]{9}" placeholder="9876543210"
                 aria-describedby="mobile-hint mobile-error">
          <p class="hint" id="mobile-hint">10 digits, starting with 6, 7, 8 or 9.</p>
          <p class="error-msg" id="mobile-error" aria-live="polite"></p>
        </div>

        <div class="field">
          <label for="password">Password <span class="req" aria-hidden="true">*</span></label>
          <div class="input-row">
            <input type="password" id="password" name="password" autocomplete="new-password"
required
                   minlength="8" maxlength="64" aria-describedby="password-rules strength-text
password-error">
            <button class="button secondary small" id="password-toggle" type="button" aria-
pressed="false"
                    aria-controls="password confirm">Show</button>
          </div>
          <div class="strength-bar" aria-hidden="true"><span id="strength-fill" data-
level="0"></span></div>
          <p class="strength-text" id="strength-text" aria-live="polite">Password strength: not
entered</p>
          <ul class="rules" id="password-rules">
            <li data-rule="length">At least 8 characters</li>
            <li data-rule="lower">One lowercase letter</li>
            <li data-rule="upper">One uppercase letter</li>
            <li data-rule="digit">One digit</li>
            <li data-rule="special">One special character (for example ! @ # $ %)</li>
          </ul>
          <p class="error-msg" id="password-error" aria-live="polite"></p>
        </div>

        <div class="field">
          <label for="confirm">Confirm password <span class="req" aria-
hidden="true">*</span></label>
          <input type="password" id="confirm" name="confirm" autocomplete="new-password"
required
                 maxlength="64" aria-describedby="confirm-error">
          <p class="error-msg" id="confirm-error" aria-live="polite"></p>
        </div>

        <div class="field-row">
          <div class="field">
```

```
            <label for="course">Course <span class="req" aria-hidden="true">*</span></label>
            <select id="course" name="course" required aria-describedby="course-error">
              <option value="">Select a course</option>
              <option value="Computer Engineering">Computer Engineering</option>
              <option value="Information Technology">Information Technology</option>
              <option value="Electronics and Communication">Electronics and
Communication</option>
              <option value="Mechanical Engineering">Mechanical Engineering</option>
              <option value="Civil Engineering">Civil Engineering</option>
            </select>
            <p class="error-msg" id="course-error" aria-live="polite"></p>
          </div>

          <div class="field">
            <label for="year">Year <span class="req" aria-hidden="true">*</span></label>
            <select id="year" name="year" required aria-describedby="year-error">
              <option value="">Select a year</option>
              <option value="1">First year</option>
              <option value="2">Second year</option>
              <option value="3">Third year</option>
              <option value="4">Fourth year</option>
            </select>
            <p class="error-msg" id="year-error" aria-live="polite"></p>
          </div>
        </div>

        <fieldset class="field choice-group" id="gender-group" aria-describedby="gender-error">
          <legend>Gender <span class="req" aria-hidden="true">*</span></legend>
          <label class="choice"><input type="radio" id="gender-female" name="gender"
value="Female" required> Female</label>
          <label class="choice"><input type="radio" id="gender-male" name="gender"
value="Male"> Male</label>
          <label class="choice"><input type="radio" id="gender-other" name="gender"
value="Other"> Other</label>
          <label class="choice"><input type="radio" id="gender-na" name="gender" value="Prefer
not to say"> Prefer not to say</label>
          <p class="error-msg" id="gender-error" aria-live="polite"></p>
        </fieldset>

        <!-- Advanced extension: custom CAPTCHA drawn on a canvas -->
        <div class="field captcha-field">
          <label for="captcha">Security check <span class="req" aria-
hidden="true">*</span></label>
          <div class="captcha-row">
            <canvas id="captcha-canvas" width="170" height="52" role="img"
                    aria-label="CAPTCHA image with 5 letters and digits"></canvas>
            <button class="button secondary small" id="captcha-refresh" type="button">New
code</button>
          </div>
          <input type="text" id="captcha" name="captcha" maxlength="5" autocomplete="off"
autocapitalize="characters"
                 spellcheck="false" required aria-describedby="captcha-hint captcha-error">
          <p class="hint" id="captcha-hint">Type the 5 characters shown in the image (not case-
sensitive).</p>
          <p class="error-msg" id="captcha-error" aria-live="polite"></p>
        </div>

        <div class="field">
          <label class="choice">
            <input type="checkbox" id="terms" name="terms" required aria-describedby="terms-
error">
            I accept the <a href="about.html">terms and conditions</a> <span class="req" aria-
hidden="true">*</span>
          </label>
          <p class="error-msg" id="terms-error" aria-live="polite"></p>
        </div>

        <div class="form-actions">
          <button class="button" type="submit">Register</button>
```

```
          <button class="button secondary" type="reset" id="reset-btn">Reset</button>
        </div>
      </form>

      <!-- Testing helper: loads the sample data used in the test-case table -->
      <details class="test-helper">
        <summary>Testing helper (for lab demonstration only)</summary>
        <p class="muted">Fill the form with sample data, then press Register.</p>
        <button class="button secondary small" id="fill-valid" type="button">Fill valid
sample</button>
        <button class="button secondary small" id="fill-invalid" type="button">Fill invalid
sample</button>
      </details>
    </section>
  </main>

  <footer class="site-footer">
    <div class="container">
      <p>&copy; 2026 StudentHub | Registration Form with Client-Side Validation</p>
    </div>
  </footer>
</body>
</html>
```

3. Additional CSS (append to css/style.css)

```
/* ===== Practical 5 additions: append to css/style.css ===== */

/* Extra colour tokens for form feedback (both themes keep readable contrast) */
:root {
  --danger: #b42318;
  --danger-bg: #fdecea;
  --success: #17692e;
  --success-bg: #e6f4ea;
  --field-border: #5b6b7f;
}
:root[data-theme="dark"] {
  --danger: #ff9b8f;
  --danger-bg: #3b1d1d;
  --success: #7ee2a0;
  --success-bg: #12301c;
  --field-border: #94a3b8;
}

.visually-hidden { position: absolute; width: 1px; height: 1px; margin: -1px; padding: 0;
overflow: hidden; clip: rect(0 0 0 0); white-space: nowrap; border: 0; }

/* Form layout */
.form-panel { max-width: 720px; margin-inline: auto; }
.field { margin-bottom: 1.1rem; border: 0; padding: 0; min-width: 0; }
.field-row { display: grid; grid-template-columns: 1fr; gap: 0 1rem; }
label, legend { display: block; font-weight: bold; margin-bottom: .3rem; padding: 0; }
.req { color: var(--danger); }
input[type="text"], input[type="email"], input[type="tel"], input[type="password"], select {
  width: 100%; padding: .6rem .7rem; font: inherit; color: var(--text); background: var(--
surface);
  border: 2px solid var(--field-border); border-radius: .4rem; transition: border-color .2s
ease, box-shadow .2s ease;
}
input:focus-visible, select:focus-visible { outline: 3px solid #f5b700; outline-offset: 1px; }
.hint { margin: .25rem 0 0; font-size: .875rem; color: var(--muted); }
.error-msg { margin: .3rem 0 0; min-height: 1.2em; font-size: .9rem; font-weight: bold; color:
var(--danger); }
.error-msg:not(:empty)::before { content: "\26A0  "; }          /* warning sign: not colour
alone */
.has-error input, .has-error select { border-color: var(--danger); background: var(--danger-
bg); }
.is-valid input:not([type="radio"]):not([type="checkbox"]), .is-valid select { border-color:
var(--success); }
```

```
/* Radio and checkbox groups */
.choice-group { border: 0; }
.choice { display: flex; align-items: center; gap: .5rem; font-weight: normal; margin: .25rem
0; cursor: pointer; }
.choice input { width: 1.15rem; height: 1.15rem; flex: none; }
.choice-group .choice { display: inline-flex; margin-right: 1.25rem; }

/* Password row, strength meter and rule list */
.input-row { display: flex; gap: .5rem; align-items: stretch; }
.button.small { padding: .4rem .8rem; font-size: .9rem; white-space: nowrap; }
.strength-bar { height: .6rem; margin-top: .5rem; background: var(--border); border-radius:
.3rem; overflow: hidden; }
.strength-bar span { display: block; height: 100%; width: 0; transition: width .3s ease,
background-color .3s ease; }
.strength-bar span[data-level="1"] { width: 25%; background: #d93025; }
.strength-bar span[data-level="2"] { width: 50%; background: #e37400; }
.strength-bar span[data-level="3"] { width: 75%; background: #1a73e8; }
.strength-bar span[data-level="4"] { width: 100%; background: #188038; }
.strength-text { margin: .3rem 0 0; font-weight: bold; }
.rules { list-style: none; margin: .5rem 0 0; padding: 0; font-size: .9rem; color: var(--
muted); }
.rules li::before { content: "\2717  "; }                          /* cross mark */
.rules li::after { content: " (" attr(data-state) ")"; }
.rules li.met { color: var(--success); }
.rules li.met::before { content: "\2713  "; }                      /* check mark */

/* Error summary and success message */
.error-summary { border: 2px solid var(--danger); background: var(--danger-bg); color: var(--
text); border-radius: .5rem; padding: .75rem 1rem; margin-bottom: 1.25rem; }
.error-summary h2 { margin: 0 0 .5rem; font-size: 1.1rem; color: var(--danger); }
.error-summary ul { margin: 0; padding-left: 1.2rem; }
.error-summary a { color: var(--text); }
.alert { border-radius: .5rem; padding: .75rem 1rem; margin-bottom: 1.25rem; }
.alert.success { border: 2px solid var(--success); background: var(--success-bg); color: var(--
text); }
.alert.error { border: 2px solid var(--danger); background: var(--danger-bg); color: var(--
text); }

/* Canvas CAPTCHA */
.captcha-row { display: flex; flex-wrap: wrap; gap: .75rem; align-items: center; margin-bottom:
.5rem; }
#captcha-canvas { border: 2px solid var(--field-border); border-radius: .4rem; max-width: 100%;
}
#captcha { max-width: 12rem; text-transform: uppercase; letter-spacing: .15em; }

.form-actions { display: flex; flex-wrap: wrap; gap: .75rem; margin-top: 1.5rem; }
.test-helper { margin-top: 1.75rem; border-top: 1px dashed var(--border); padding-top: 1rem; }
.test-helper summary { cursor: pointer; font-weight: bold; }

@media (min-width: 600px) {
  .field-row { grid-template-columns: 1fr 1fr; }
}
```

4. JavaScript validation (save as js/register.js)

```
'use strict';

/* ============================================================
   Practical 5: Registration form validation (client side)
   Part 1 - pure validation logic (no DOM, easy to test)
   Part 2 - DOM wiring (events, error messages, strength meter)
   Part 3 - Canvas CAPTCHA (advanced extension)
   ============================================================ */

/* ---------- Part 1A: Regular expressions ---------- */
const PATTERNS = {
```

```
  // starts with a letter; letters, space, dot, apostrophe, hyphen; 2-50 characters
  name: /^[A-Za-z][A-Za-z .'-]{1,49}$/,
  // no consecutive dots, no leading/trailing dot, domain with a 2+ letter TLD
  email: /^[A-Za-z0-9]+(?:[._%+-][A-Za-z0-9]+)*@[A-Za-z0-9]+(?:-[A-Za-z0-9]+)*(?:\.[A-Za-z0-
9]+(?:-[A-Za-z0-9]+)*)*\.[A-Za-z]{2,}$/,
  // Indian mobile number: 10 digits starting with 6-9
  mobile: /^[6-9]\d{9}$/,
  // password building blocks
  lower: /[a-z]/,
  upper: /[A-Z]/,
  digit: /\d/,
  special: /[^A-Za-z0-9\s]/,
  whitespace: /\s/,
  repeated: /(.)\1{2,}/,                          // aaa, 111 ...
  common: /password|qwerty|123456|admin|welcome|letmein/i
};

/* ---------- Part 1B: Password strength ---------- */
function checkPassword(pw) {
  const rules = {
    length: pw.length >= 8,
    lower: PATTERNS.lower.test(pw),
    upper: PATTERNS.upper.test(pw),
    digit: PATTERNS.digit.test(pw),
    special: PATTERNS.special.test(pw)
  };
  let score = Object.values(rules).filter(Boolean).length;      // 0 - 5
  if (pw.length >= 12) score += 1;                              // bonus for long passwords
  if (PATTERNS.repeated.test(pw)) score -= 2;                   // penalty: aaa, 111
  if (PATTERNS.common.test(pw)) score = Math.min(score, 2);     // penalty: common words
  score = Math.max(0, Math.min(6, score));

  let level = 0;
  let label = 'not entered';
  if (pw.length > 0) {
    if (score <= 2) { level = 1; label = 'Weak'; }
    else if (score === 3) { level = 2; label = 'Fair'; }
    else if (score <= 5) { level = 3; label = 'Good'; }
    else { level = 4; label = 'Strong'; }
  }
  return { rules, score, level, label };
}

/* ---------- Part 1C: Field validators ----------
   Each validator returns '' when valid, otherwise a friendly message.
   ctx carries values needed from other fields (password, captcha code). */
const validators = {
  fullname(value) {
    const v = value.trim();
    if (!v) return 'Enter your full name.';
    if (v.length < 2) return 'Name must have at least 2 characters.';
    if (v.length > 50) return 'Name must not be longer than 50 characters.';
    if (!PATTERNS.name.test(v) || !/[A-Za-z].*[A-Za-z]/.test(v)) {
      return 'Use letters, spaces, dots, apostrophes or hyphens only (no digits or symbols).';
    }
    return '';
  },

  email(value) {
    const v = value.trim();
    if (!v) return 'Enter your email address.';
    if (v.length > 254) return 'Email address is too long.';
    if (!PATTERNS.email.test(v)) return 'Enter a valid email address, for example
name@example.com.';
    return '';
  },

  mobile(value) {
    const v = value.trim();
```

```
    if (!v) return 'Enter your mobile number.';
    if (/\D/.test(v)) return 'Use digits only: no spaces, dashes or +91.';
    if (v.length !== 10) return 'Mobile number must have exactly 10 digits.';
    if (!PATTERNS.mobile.test(v)) return 'Mobile number must start with 6, 7, 8 or 9.';
    return '';
  },

  password(value) {
    if (!value) return 'Create a password.';
    if (PATTERNS.whitespace.test(value)) return 'Password must not contain spaces.';
    const { rules } = checkPassword(value);
    const missing = [];
    if (!rules.length) missing.push('at least 8 characters');
    if (!rules.lower) missing.push('a lowercase letter');
    if (!rules.upper) missing.push('an uppercase letter');
    if (!rules.digit) missing.push('a digit');
    if (!rules.special) missing.push('a special character');
    if (missing.length) return 'Password needs ' + missing.join(', ') + '.';
    if (PATTERNS.common.test(value)) return 'Password contains a very common word. Choose
something less guessable.';
    return '';
  },

  confirm(value, ctx = {}) {
    if (!value) return 'Re-enter your password to confirm it.';
    if (value !== ctx.password) return 'Passwords do not match.';
    return '';
  },

  course(value) {
    return value ? '' : 'Select your course.';
  },

  year(value) {
    return /^[1-4]$/.test(value) ? '' : 'Select your year of study.';
  },

  gender(value) {
    return value ? '' : 'Select one option for gender.';
  },

  captcha(value, ctx = {}) {
    const v = value.trim();
    if (!v) return 'Type the characters shown in the image.';
    if (v.toUpperCase() !== String(ctx.captcha || '').toUpperCase()) return 'The characters do
not match. Try again or get a new code.';
    return '';
  },

  terms(checked) {
    return checked ? '' : 'You must accept the terms and conditions to register.';
  }
};

/* Allow the pure logic to be loaded by a Node test file */
if (typeof module !== 'undefined' && module.exports) {
  module.exports = { PATTERNS, checkPassword, validators };
}

/* ============================================================
   Part 2: DOM wiring (runs only in a browser)
   ============================================================ */
if (typeof document !== 'undefined') {
  document.addEventListener('DOMContentLoaded', initRegistrationForm);
}

function initRegistrationForm() {
```

```
  const form = document.querySelector('#register-form');
  if (!form) return;

  const summary = document.querySelector('#error-summary');
  const summaryList = document.querySelector('#error-list');
  const successBox = document.querySelector('#success-box');
  const captcha = { code: '' };
  const touched = new Set();

  /* Field map: which element shows the error, which element gets aria-invalid,
     and where the summary link should point. */
  const fieldNames = ['fullname', 'email', 'mobile', 'password', 'confirm', 'course', 'year',
'gender', 'captcha', 'terms'];
  const labels = {
    fullname: 'Full name', email: 'Email', mobile: 'Mobile number', password: 'Password',
    confirm: 'Confirm password', course: 'Course', year: 'Year', gender: 'Gender',
    captcha: 'Security check', terms: 'Terms and conditions'
  };
  const focusTarget = (name) => (name === 'gender' ? document.querySelector('#gender-female') :
document.querySelector('#' + name));

  const getValue = (name) => {
    if (name === 'gender') {
      const checked = form.querySelector('input[name="gender"]:checked');
      return checked ? checked.value : '';
    }
    if (name === 'terms') return form.elements.terms.checked;
    return form.elements[name].value;
  };

  const context = () => ({ password: form.elements.password.value, captcha: captcha.code });

  /* ---- Show / clear an error next to its field ---- */
  function setError(name, message) {
    const errorEl = document.querySelector('#' + name + '-error');
    errorEl.textContent = message;
    const wrapper = errorEl.closest('.field');
    wrapper.classList.toggle('has-error', Boolean(message));
    wrapper.classList.toggle('is-valid', !message && touched.has(name));
    if (name !== 'gender' && name !== 'terms') {
      const input = document.querySelector('#' + name);
      input.setAttribute('aria-invalid', String(Boolean(message)));
    }
  }

  /* ---- Validate one field and update the UI ---- */
  function validateField(name) {
    const message = validators[name](getValue(name), context());
    setError(name, message);
    return message;
  }

  /* ---- Password strength meter ---- */
  function updateStrength() {
    const pw = form.elements.password.value;
    const result = checkPassword(pw);
    const fill = document.querySelector('#strength-fill');
    fill.dataset.level = String(result.level);
    document.querySelector('#strength-text').textContent = 'Password strength: ' +
result.label;
    document.querySelectorAll('#password-rules li').forEach((li) => {
      const met = result.rules[li.dataset.rule];
      li.classList.toggle('met', met);
      // text is added as well as colour so the state is not conveyed by colour alone
      li.dataset.state = met ? 'Met' : 'Not met';
    });
  }
```

```
  /* ---- Real-time validation (intermediate extension) ----
     A field is validated when the user leaves it (blur). After that,
     it is re-checked on every keystroke so the message disappears as soon as it is fixed. */
  fieldNames.forEach((name) => {
    if (name === 'gender' || name === 'terms' || name === 'course' || name === 'year') return;
    const input = document.querySelector('#' + name);
    input.addEventListener('blur', () => { touched.add(name); validateField(name); });
    input.addEventListener('input', () => { if (touched.has(name)) validateField(name); });
  });

  ['course', 'year'].forEach((name) => {
    document.querySelector('#' + name).addEventListener('change', () => { touched.add(name);
validateField(name); });
  });
  form.querySelectorAll('input[name="gender"]').forEach((radio) => {
    radio.addEventListener('change', () => { touched.add('gender'); validateField('gender');
});
  });
  form.elements.terms.addEventListener('change', () => { touched.add('terms');
validateField('terms'); });

  form.elements.password.addEventListener('input', () => {
    updateStrength();
    if (touched.has('confirm')) validateField('confirm');   // keep confirm in sync
  });
  form.elements.password.addEventListener('keyup', updateStrength);   // keyup as well as input
(real-time feedback)

  /* ---- Show / hide password ---- */
  const toggle = document.querySelector('#password-toggle');
  toggle.addEventListener('click', () => {
    const show = form.elements.password.type === 'password';
    form.elements.password.type = show ? 'text' : 'password';
    form.elements.confirm.type = show ? 'text' : 'password';
    toggle.textContent = show ? 'Hide' : 'Show';
    toggle.setAttribute('aria-pressed', String(show));
  });

  /* ---- Submit ---- */
  form.addEventListener('submit', (event) => {
    event.preventDefault();
    successBox.hidden = true;

    const errors = [];
    fieldNames.forEach((name) => {
      touched.add(name);
      const message = validateField(name);
      if (message) errors.push({ name, message });
    });

    if (errors.length) {
      showSummary(errors);
      return;
    }
    summary.hidden = true;
    showSuccess();
  });

  function showSummary(errors) {
    summaryList.textContent = '';
    errors.forEach(({ name, message }) => {
      const li = document.createElement('li');
      const link = document.createElement('a');
      link.href = '#' + focusTarget(name).id;
      link.textContent = labels[name] + ': ' + message;
      link.addEventListener('click', (event) => {
        event.preventDefault();
        focusTarget(name).focus();
```

```
      });
      li.appendChild(link);
      summaryList.appendChild(li);
    });
    summary.hidden = false;
    summary.focus();                                    // screen readers announce the summary
    summary.scrollIntoView({ behavior: 'smooth', block: 'center' });
  }

  function showSuccess() {
    const data = {
      name: form.elements.fullname.value.trim(),
      email: form.elements.email.value.trim(),
      mobile: form.elements.mobile.value.trim(),
      course: form.elements.course.value,
      year: form.elements.year.value
    };
    successBox.textContent = '';
    const title = document.createElement('strong');
    title.textContent = 'Registration successful. ';
    successBox.appendChild(title);
    successBox.appendChild(document.createTextNode(
      'Welcome, ' + data.name + '. Course: ' + data.course + ', year ' + data.year +
      '. A confirmation will be sent to ' + data.email + '. (Demonstration only: nothing was
sent to a server.)'
    ));
    successBox.hidden = false;
    successBox.focus();
    form.reset();
    resetUi();
  }

  /* ---- Reset ---- */
  function resetUi() {
    touched.clear();
    fieldNames.forEach((name) => setError(name, ''));
    form.querySelectorAll('.is-valid, .has-error').forEach((el) => el.classList.remove('is-
valid', 'has-error'));
    summary.hidden = true;
    updateStrength();
    form.elements.password.type = 'password';
    form.elements.confirm.type = 'password';
    toggle.textContent = 'Show';
    toggle.setAttribute('aria-pressed', 'false');
    drawCaptcha();
  }
  form.addEventListener('reset', () => { successBox.hidden = true; setTimeout(resetUi, 0); });

  /* ============================================================
     Part 3: Canvas CAPTCHA (advanced extension)
     ============================================================ */
  const canvas = document.querySelector('#captcha-canvas');
  function randomInt(max) {
    const buffer = new Uint32Array(1);
    window.crypto.getRandomValues(buffer);
    return buffer[0] % max;
  }

  function drawCaptcha() {
    const ctx = canvas.getContext('2d');
    const chars = 'ABCDEFGHJKLMNPQRSTUVWXYZ23456789';   // no 0/O or 1/I to avoid confusion
    let code = '';
    for (let i = 0; i < 5; i += 1) code += chars[randomInt(chars.length)];
    captcha.code = code;

    ctx.fillStyle = '#eef3fb';
    ctx.fillRect(0, 0, canvas.width, canvas.height);

    // noise lines and dots
```

```
    for (let i = 0; i < 6; i += 1) {
      ctx.strokeStyle = 'rgba(23, 78, 166, ' + (0.25 + Math.random() * 0.3) + ')';
      ctx.lineWidth = 1 + Math.random() * 1.5;
      ctx.beginPath();
      ctx.moveTo(Math.random() * canvas.width, Math.random() * canvas.height);
      ctx.bezierCurveTo(Math.random() * canvas.width, Math.random() * canvas.height,
                        Math.random() * canvas.width, Math.random() * canvas.height,
                        Math.random() * canvas.width, Math.random() * canvas.height);
      ctx.stroke();
    }
    for (let i = 0; i < 40; i += 1) {
      ctx.fillStyle = 'rgba(16, 35, 63, ' + Math.random() * 0.4 + ')';
      ctx.fillRect(Math.random() * canvas.width, Math.random() * canvas.height, 2, 2);
    }

    // characters with random rotation and vertical offset
    ctx.textBaseline = 'middle';
    ctx.font = 'bold 30px "Courier New", monospace';
    const step = canvas.width / (code.length + 1);
    [...code].forEach((ch, i) => {
      ctx.save();
      ctx.translate(step * (i + 1), canvas.height / 2 + (Math.random() * 10 - 5));
      ctx.rotate((Math.random() - 0.5) * 0.6);
      ctx.fillStyle = '#10233f';
      ctx.fillText(ch, -10, 0);
      ctx.restore();
    });
  }

  document.querySelector('#captcha-refresh').addEventListener('click', () => {
    drawCaptcha();
    form.elements.captcha.value = '';
    setError('captcha', '');
    form.elements.captcha.focus();
  });

  /* ---- Testing helper: fills sample data used in the test-case table ---- */
  document.querySelector('#fill-valid').addEventListener('click', () => {
    form.elements.fullname.value = 'Riya Patel';
    form.elements.email.value = 'riya.patel@example.com';
    form.elements.mobile.value = '9876543210';
    form.elements.password.value = 'Studen#Hub26';
    form.elements.confirm.value = 'Studen#Hub26';
    form.elements.course.value = 'Computer Engineering';
    form.elements.year.value = '2';
    document.querySelector('#gender-female').checked = true;
    form.elements.captcha.value = captcha.code;
    form.elements.terms.checked = true;
    updateStrength();
  });
  document.querySelector('#fill-invalid').addEventListener('click', () => {
    form.elements.fullname.value = 'R2';
    form.elements.email.value = 'riya@@example';
    form.elements.mobile.value = '12345';
    form.elements.password.value = 'password';
    form.elements.confirm.value = 'pass123';
    form.elements.course.value = '';
    form.elements.year.value = '';
    form.querySelectorAll('input[name="gender"]').forEach((r) => { r.checked = false; });
    form.elements.captcha.value = 'XXXXX';
    form.elements.terms.checked = false;
    updateStrength();
  });

  updateStrength();
  drawCaptcha();
}
```

7. Extensions

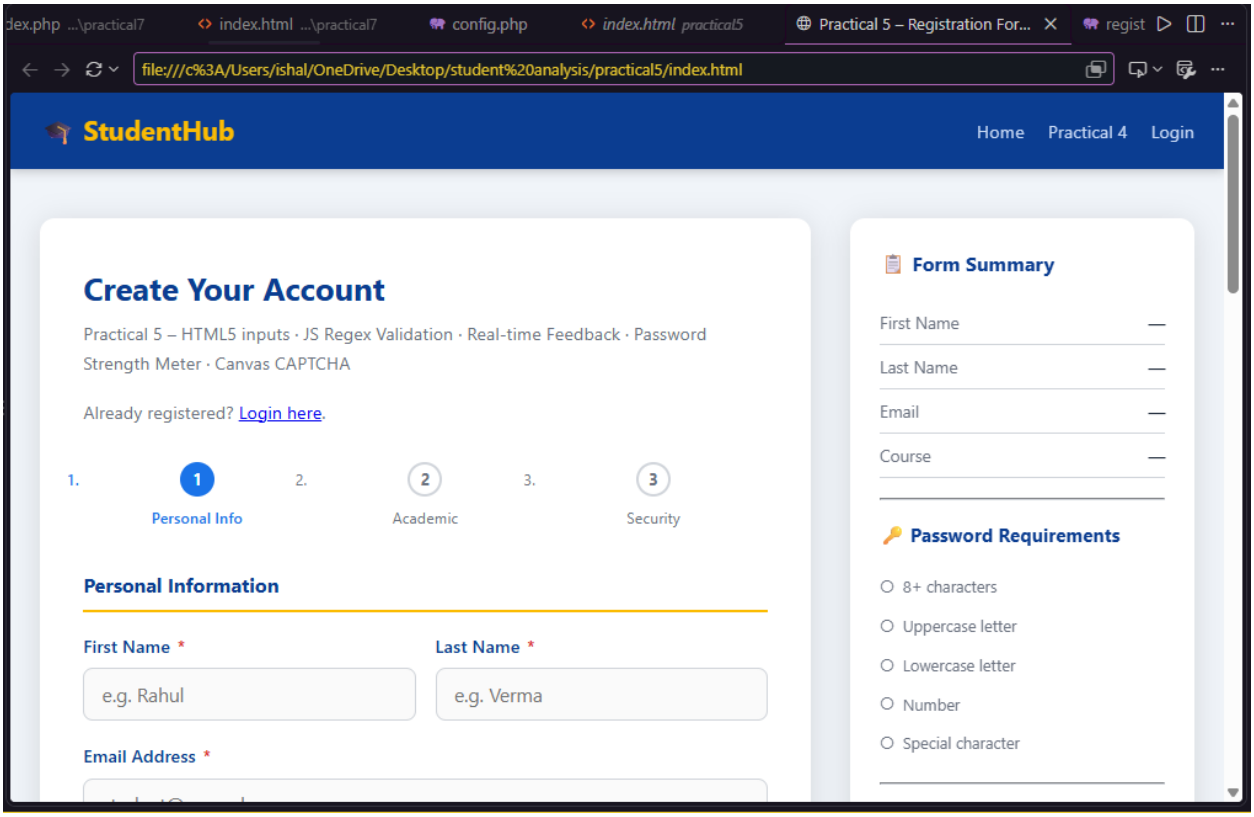
Intermediate: Real-time validation is added with blur, input, keyup, and change event listeners. Each field is validated when the user leaves it, and then re-validated on every keystroke so that the error clears immediately. When the password changes, the confirm-password field is checked again if it has already been touched.

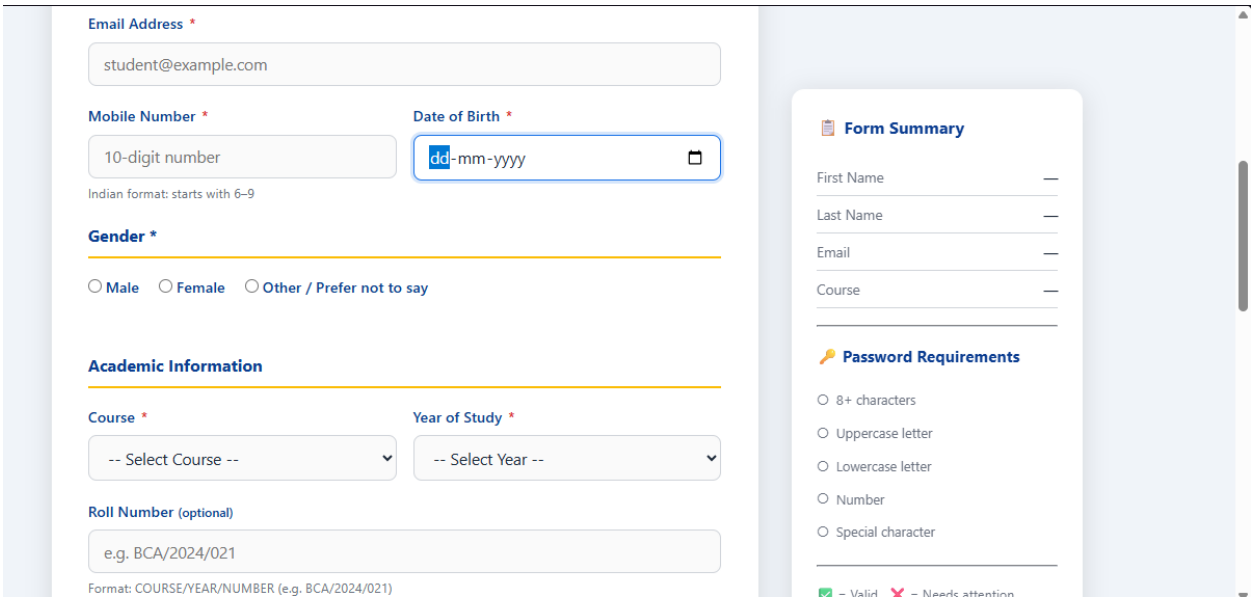
Advanced: A custom CAPTCHA is drawn with the Canvas API (fillRect, bezierCurveTo, rotate, fillText). Characters are chosen with crypto.getRandomValues(), and confusing characters (0, O, 1, I) are excluded. The code is regenerated after a successful submission or when the user presses New code.

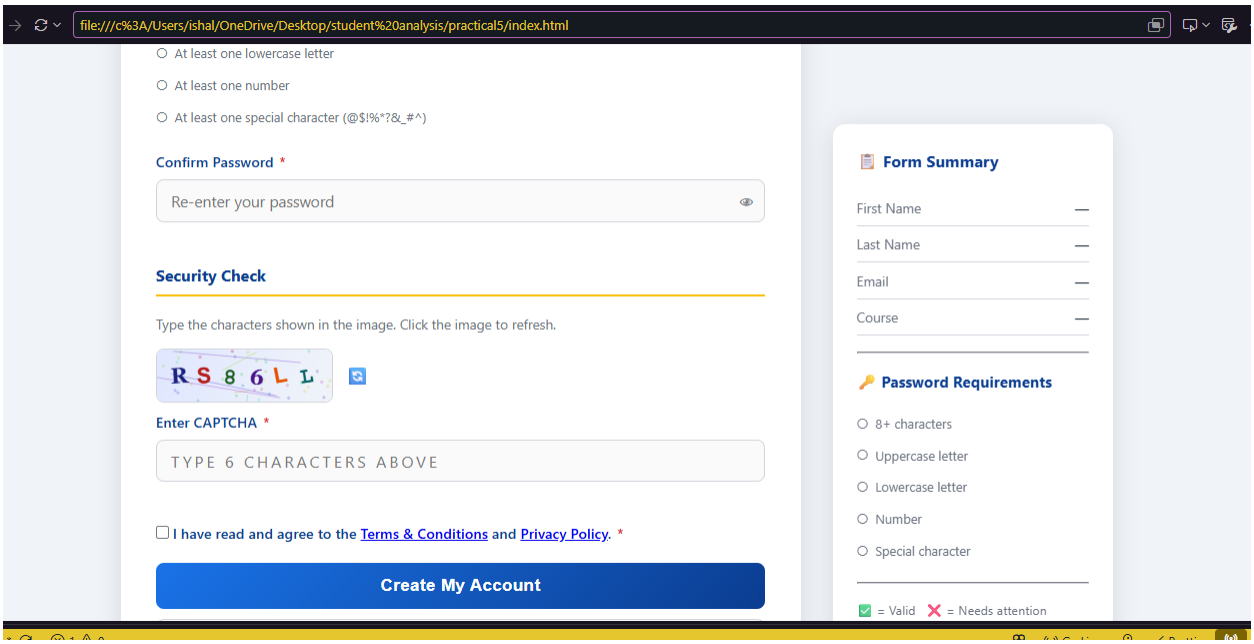
C. Evidence of Work (Screenshots)

Run the page in a browser and replace each frame below with your own screenshot. Also include the VS Code project structure and the Console test

Registration form — empty form on load (desktop, light theme)







D. Form Validation and Accessibility Checklist

☐ Correct input types (text, email, tel, password) and attributes (required, pattern, autocomplete) are used.

☐ Every control has a visible label connected with for/id.

☐ Radio buttons are grouped with fieldset and legend.

☐ Regular expressions validate name, email, and mobile number.

☐ Error messages appear next to the relevant fields and are written in plain language.

☐ aria-invalid and aria-describedby are updated when errors appear and disappear.

☐ An error summary links to each problem and receives focus on submit.

☐ Password strength is shown as a bar, as text, and as a rules checklist.

☐ Confirm password must match the password.

☐ Course, year, gender, and terms are required.

☐ Validation runs in real time (blur, input, keyup, change).

☐ The CAPTCHA can be refreshed and is checked on submit.

☐ The form can be completed using the keyboard only and focus is clearly visible.

☐ Errors are not shown by colour alone (text and a warning sign are present).

☐ No inline CSS or inline JavaScript event attributes are used.

E. Challenges Faced

1. Writing an email regular expression that rejects double dots and missing top-level domains without becoming unreadable.

2. Deciding when to show errors: validating on blur first avoids showing errors before the user has finished typing.

3. Keeping the confirm-password field in sync when the password changes later.

4. Announcing errors to screen readers without repeating them too often.

5. Making the CAPTCHA readable while still adding noise, and keeping it keyboard accessible.

F. Learning Outcomes

1. Used correct HTML5 input types and attributes to build a complete registration form.

2. Wrote and tested regular expressions for names, emails, mobile numbers, and passwords.

3. Displayed clear, friendly validation messages near each field and in an error summary.

4. Built a password strength meter and applied real-time validation with several event types.

5. Designed a form that can be used with a keyboard and a screen reader.

G. Viva Questions

What does the novalidate attribute do?

Answer: It turns off the browser’s built-in validation pop-ups so that the page can show its own messages.

Which input type shows a numeric keypad on phones?

Answer: type="tel" (with inputmode="numeric").

What does /^[6-9]\d{9}$/ match?

Answer: A ten-digit number that starts with 6, 7, 8, or 9.

Why are ^ and $ used in validation patterns?

Answer: They force the whole value to match, not just a part of it.

What is the difference between test() and match()?

Answer: test() returns true or false; match() returns the matched text or null.

What is aria-live used for here?

Answer: It makes screen readers announce an error message when its text changes.

Why is aria-describedby used?

Answer: It connects an input to its hint and error text so they are read with the field.

Why validate on blur and then on input?

Answer: To avoid showing errors while the user is still typing, and to remove them quickly once fixed.

How is confirm password checked?

Answer: By comparing its value with the password field using the strict equality operator (!==).

Why is a lookahead such as (?=.*\d) not used in the password check?

Answer: Separate small tests for each rule give an individual message for every missing rule and drive the checklist.

Why should colour not be the only error indicator?

Answer: Users with colour-vision deficiency may not notice it, so text and an icon are added.

Why must the server validate again?

Answer: Client-side checks can be bypassed; the server is the last line of defence (Practical 7).

H. Result

A student registration form was implemented using HTML5 input types and JavaScript regular-expression validation. Errors are displayed beside each field and in a summary, password strength is measured and displayed, validation runs in real time, a canvas CAPTCHA is included, and the form is usable by keyboard and screen reader. Valid and invalid test inputs were checked, and the test table and screenshots demonstrate the final result.

Faculty Remarks

(Keep this section blank for faculty.)